

GUIDE DE DÉMARRAGE PAS-À-PAS

YahriaCad

CAD PCB piloté par l'IA — de l'installation au premier export

Ce guide couvre tout le parcours d'utilisation de YahriaCad : lancement de la stack (Docker ou natif), création d'un premier projet, routage interactif et IA, benchmark Arena A* vs RL, outils de vérification, collaboration CRDT et exports fabricants (Gerber, ODB++, STEP, BOM).

RÉPERTOIRE

assihervey-coder/YahriaCad

VERSION

v0.1.0

DATE

Septembre 2026

Démarrer la stack en 2 minutes

YahriaCad se compose de trois services : un **backend Go** (REST + WebSocket, port 8080), un **moteur IA Python/gRPC** (port interne 50051) et un **frontend Next.js** (port 3000), avec PostgreSQL en option pour la persistance. Deux voies de lancement existent, selon que vous voulez la stack complète sans toucher à votre environnement, ou un mode développement itératif.

Voie A — Docker (recommandée)

Une seule commande construit les trois images et démarre l'intégralité de la stack : postgres 16, le moteur IA, le backend (JWT activé) et le frontend SSR. Les volumes `pgdata` et `backenddata` persistent entre les redémarrages.

```
# stack complète : build + démarrage détaché$ make docker-up
# équivalent manuel :$ docker compose -f docker/docker-compose.yml up -d --build
# arrêt propre :$ make docker-down
```

Voie B — Développement natif

Trois terminaux (ou trois processus en arrière-plan). Le mode natif démarre *sans PostgreSQL* : le backend bascule alors automatiquement sur son adaptateur mémoire — parfait pour itérer. Le moteur IA reste optionnel : sans lui, le routage retombe sur le solveur **A*** déterministe intégré au backend.

```
# 1. dépendances (Go modules, pip, npm)$ make deps
# 2. trois services, trois terminaux$ make run-backend    # API Go .....
http://localhost:8080$ make run-ai                    # moteur gRPC ..... localhost:50051$
make run-frontend  # interface ..... http://localhost:3000
```

Vérifier la santé de la stack

```
$ curl -s localhost:8080/healthz | jq
{
  "status": "ok", "version": "0.1.0",
  "ai_engine": "ok", "ai_device": "cpu",
  "ai_model_loaded": false, "database": "postgres"
}
```

Les champs `ai_engine`, `ai_device` et `ai_model_loaded` indiquent l'état du runtime RL ; `database` vaut `postgres` ou `memory`. Si `ai_engine` vaut `unreachable`, l'API reste utilisable mais le routage passera par A* (voir dépannage §09).

Première connexion

En Docker, l'authentification JWT est activée avec des identifiants de démonstration `admin:admin` (à surcharger via un fichier `.env`). Sans configuration d'authentification (mode natif par défaut), la page de login propose « Continuer sans connexion » et l'API reste ouverte.

02 · VUE D'ENSEMBLE

Architecture en un coup d'œil

Comprendre les flux évite 90 % des erreurs de configuration. Le frontend ne parle jamais directement au moteur IA : tout passe par le backend Go, qui orchestre, valide et journalise. Les traitements longs (placement, routage, optimisation, benchmark) sont des **jobs asynchrones** : le POST renvoie immédiatement un `job_id`, et la progression arrive soit par polling REST, soit par le canal WebSocket `/ws/v1/progress`.

Frontend Next.js

Éditeur schéma & layout, panneau IA, collab CRDT
:3000

REST + WEBSOCKET /WS/V1/PROGRESS

Backend Go

API REST, jobs, DRC/ERC, exports, auth JWT, /metrics
:8080

GRPC (PCB.V1) · SQL · FICHIERS

Moteur IA Python

RL PPO (PyTorch) + fallback
A*, hot reload
gRPC :50051

PostgreSQL 16

Persistance des projets (sinon
mémoire)
:5432

/data

Exports Gerber, ODB++, STEP,
BOM
YAHRIACAD_DATA_DIR

Côté code, le backend suit une architecture hexagonale stricte (`domain` → `application` → `infrastructure`) : le contrat gRPC `shared/types/pcb.proto` (package `yahriacad.pcb.v1`) est partagé avec le moteur IA via les stubs régénérables par `make proto`. Cette séparation rend le moteur IA remplaçable et le fallback A* transparent pour le client.

Premier projet de A à Z

Le parcours minimal complet tient en cinq étapes. Tout est réalisable à l'interface (recommandé pour un premier contact) ou par l'API pour scripter.

1 Créer le projet — Project Manager

Ouvrez `localhost:3000` → page **Project Manager** → « Nouveau projet ». Par API : `POST /api/v1/projects` avec nom et description.

2 Importer la netlist

`POST /api/v1/projects/{id}/import` accepte la description de schéma (composants, broches, nets). Le parseur remplit le domaine, prêt pour le placement.

3 Contrôler le schéma — Schematic Editor

Vérifiez nets et composants ; lancez un **ERC** pour attraper tôt les erreurs de connectivité (`POST .../erc`).

4 Ouvrir le PCB Layout

La page **pcb-layout** affiche le board, la liste des nets avec leur classe, le panneau IA et la barre Magic. C'est ici que se passent placement, routage et vérifications des sections suivantes.

5 (Option) Carte cauchemar en 1 clic

Pour tester toute la chaîne sans données : `POST /api/v1/demo/nightmare` génère un projet volontairement pathologique — tutoriel vivant de l'Auto-Healer et du Doctor.

```
# création + import en ligne de commande$ TOKEN=$(curl -s
localhost:8080/api/v1/auth/login \
  -d '{"username":"admin","password":"admin"}' | jq -r .token)
$ PID=$(curl -s -H "Authorization: Bearer $TOKEN" \
  localhost:8080/api/v1/projects -d '{"name":"demo-led"}' | jq -r .id)
```

Routage IA au quotidien

Trois gros traitements partagent le même mécanisme de job asynchrone : **placement** (`/place`), **routage global** (`/route`) et **optimisation** (`/optimize`). Chaque POST renvoie un `job_id` à interroger via `GET /api/v1/projects/{id}/jobs/{jobID}` — le frontend sonne toutes les ~800 ms avec un plafond de 60 s, ou s'abonne au WebSocket. Toutes ces routes IA sont **limitées à 2 req/s par IP** (burst 10) : au-delà, le backend répond `429` avec un en-tête `Retry-After`.

Routage interactif net par net

Dans la page **pcb-layout**, chaque net de la liste possède un bouton ⚡ : il lance le routage de ce net unique via le moteur (RL si un checkpoint est chargé, sinon A*), et la piste apparaît immédiatement avec undo/redo disponible. C'est le mode le plus agréable pour finir à la main ce que le routage global a laissé.

Classes de nets & plans de masse

- **Classes de nets** — assignez chaque net à une classe (`PUT .../netclasses/{net}`) puis fixez les règles par classe : largeur de piste, clearance, `PUT .../netclasses/{class}/rules` .
- **Plans de masse (pours)** — génération de zones cuivre connectées à un net (`POST .../pours`), listables (`GET`) et supprimables globalement ou une à une.
- **Impédance différentielle** — le bouton ⚡ **Impédance** du board appelle `POST .../impedance` et renvoie l'impédance calculée pour les paires concernées.

ENDPOINT	RÔLE
POST .../place	Placement automatique des composants (job IA)
POST .../route	Routage global du board (job IA, fallback A*)
POST .../optimize	Optimisation des pistes existantes
GET .../jobs/{jobID}	Statut / résultat d'un job asynchrone
PUT .../netclasses/{net}	Assigner un net à une classe
POST .../pours	Générer les plans de masse
POST .../impedance	Impédance (incl. paires différentielles)

Arena : A* vs RL, le match en un clic

L'Arena fait s'affronter les deux routeurs sur *le même board* : le solveur **A*** déterministe du backend contre le **routeur RL** (PPO, PyTorch) du moteur. Le benchmark produit pour chaque combattant la longueur routée, le nombre de vias et un score d'arène identique, puis un verdict et un ajustement **ELO** en tête-à-tête. Un résultat réel de smoke test : A* 146,0 mm (score 592,70) contre RL 196,5 mm / 10 vias (score 49,72) — verdict A*, ELO 1212/1188.

Lancer un match

À l'interface : bouton **⚠ A* vs RL** de la MagicBar sur la page pcb-layout. Par API : `POST /api/v1/projects/{id}/arena/benchmark` (progression sur le canal WS `arena-benchmark-<pid>`). Le classement cumulé est exposé sur `GET /api/v1/arena/leaderboard`. Sans checkpoint chargé, la route répond `503 rL_model_not_loaded` — jamais un faux match A* contre A*.

Cycle de vie du modèle RL

1

Générer les checkpoints initiaux

`make models` (script `scripts/generate-models.sh`, nécessite torch) crée les fichiers `.pt` initiaux.

2

Entraîner le routeur PPO

`make train-router STEPS=200000` lance `ai-engine/training/router/train.py` (config dans `config.yaml`, tokenizer inclus).

3

Recharger à chaud

`GET /api/v1/ai/model` inspecte le runtime (device, version, modèle chargé) ; `POST /api/v1/ai/model/reload` charge le nouveau checkpoint **sans redémarrer** le moteur.

Docker & PyTorch

Dans `docker/docker-compose.yml`, l'argument `WITH_TORCH: "false"` produit une image IA légère (routeur A* uniquement). Passez-le à `"true"` pour embarquer PyTorch CPU (~+800 Mo) et débloquent le benchmark RL en conteneur. En natif, `pip install torch` suffit.

06 • QUALITÉ & CONFORT

Vérification, analyse et collaboration

YahriaCad embarque une palette complète d'outils de vérification et de confort, tous disponibles depuis le board ou par API. Ils forment la boucle « routage → contrôle → correction automatique → export » qui fait le quotidien d'un travail de routage sérieux.

DRC & ERC /drc • /erc

Règles électriques et de fabrication : clearances, nets flottants, connectivité. L'ERC se lance dès l'import.

Thermique & SI /thermal • /si

Points chauds et intégrité du signal : analyse complémentaire avant de figer le routage.

Auto-Healer /drc/autofix

Corrige automatiquement les violations DRC triviales — le plus spectaculaire sur la carte cauchemar.

Doctor GET /doctor

Diagnostic global du board : synthèse des problèmes et pistes d'action priorisées.

DFM /dfm

Vérification manufacturabilité : largeurs, vias, anneaux — avant d'envoyer chez le fabricant.

Magic Pack /magic

Batterie d'améliorations en un clic depuis la MagicBar (nettoyage, suggestions, optimisations ciblées).

Time Machine /snapshots

Captures d'état, diff entre versions, restore : explorez des variantes de routage sans peur.

Collaboration CRDT /collab/*

Édition multi-utilisateurs : ops, état partagé, undo/redo coordonnés — la présence s'affiche en direct.

Les raccourcis clavier (composant dédié côté frontend) couvrent undo/redo, sélection et zoom ; les statistiques du projet (GET .../stats) donnent nets routés, longueur totale et historique — utile pour mesurer l'effet réel d'un passage RL.

07 • LIVRAISON

Exports fabricants

Quatre formats de sortie couvrent la chaîne complète, de la fabrication de la carte à l'assemblage. Les fichiers sont écrits dans le répertoire données (YAHRIACAD_DATA_DIR , /data en Docker) et servis par la réponse HTTP. L'export ODB++ répond en application/gzip avec un en-tête X-YahriaCad-File-Count indiquant le nombre de fichiers de l'archive.

FORMAT	ENDPOINT	POUR QUOI FAIRE
Gerber	GET .../export/gerber	Fabrication du cuivre et des masques par le pcb assembler
BOM	GET .../export/bom	Nomenclature des composants pour l'achat et l'assemblage
STEP	GET .../export/step	Modèle 3D pour l'intégration mécanique (boîtier, CAO)
ODB++	GET .../export/odbpp	Échange CAM riche (netlist, pastilles, vias, couches) en .tgz

Contrôle en aval

Avant d'exporter, enchaînez **DRC** puis **DFM** : ce sont les deux filtres qui évitent les allers-retours avec le fabricant. Un snapshot Time Machine juste avant l'export vous garde une porte de sortie propre (`POST .../snapshots`).

08 • MÉMO

Référence API rapide

Toutes les routes vivent sous `/api/v1` (base `NEXT_PUBLIC_API_URL` côté frontend). Le préfixe `.../` ci-dessous désigne `/api/v1/projects/{id}`. Auth obligatoire quand JWT est activé : en-tête `Authorization: Bearer <token>`. Les routes marquées **IA** subissent le rate limiting par IP.

ROUTE	RÔLE
<code>POST /auth/login</code> • <code>GET /auth/me</code>	Connexion JWT, profil courant (503 si auth désactivée)
<code>GET POST PUT DELETE /projects...</code>	CRUD projets + <code>/import</code> , <code>/layout</code> (GET/PUT)
<code>POST .../place</code> • <code>/route</code> • <code>/optimize IA</code>	Jobs asynchrones → <code>GET .../jobs/{jobID}</code>
<code>POST .../magic</code> • <code>/arena</code> • <code>/arena/benchmark IA</code>	Magic Pack, affrontements Arena + benchmark, leaderboard global
<code>POST .../drc/autofix</code> • <code>GET .../doctor IA</code>	Auto-Healer et diagnostic global
<code>POST .../drc</code> • <code>/erc</code> • <code>/thermal</code> • <code>/si</code> • <code>/impedance</code> • <code>/dfm</code>	Suite de vérification (synchrone)
<code>POST GET DELETE .../pours...</code> • <code>GET PUT .../netclasses...</code>	Plans de masse et classes de nets
<code>POST GET .../collab/ops</code> • <code>/state</code> • <code>/undo</code> • <code>/redo</code>	Collaboration CRDT temps réel
<code>POST GET .../snapshots</code> • <code>/{{sid}}/diff</code> • <code>/{{sid}}/restore</code>	Time Machine : capture, comparaison, restauration
<code>GET .../stats</code> • <code>GET .../export/{gerber,bom,step,odbpp}</code>	Statistiques et exports fabricants

POST /demo/nightmare IA · GET POST /ai/model(/reload)	Démo cauchemar · inspection et hot reload du modèle RL
GET /healthz · GET /metrics · WS /ws/v1/progress	Sondes système, métriques Prometheus, progression temps réel

09 · QUAND ÇA COINCe

Dépannage & configuration utile

SYMPTÔME	CAUSE PROBABLE	CORRECTIF
401 invalid_credentials	Identifiants hors <code>YAHRIACAD_AUTH_USERS</code>	Format <code>user:pass;user2:\$2a\$... (bcrypt accepté)</code>
503 auth_disabled	JWT non configuré	Comportement normal en dev : l'API est ouverte, login inutile
503 ai_unreachable	Moteur IA éteint / mauvaise adresse	Lancer <code>make run-ai</code> , vérifier <code>YAHRIACAD_AI_ADDR</code> — le routage A* reste actif
503 rl_model_not_loaded	Pas de checkpoint chargé	<code>make models</code> puis <code>POST /ai/model/reload</code>
429 + Retry-After	Rate limit IA dépassé (2 rps/IP)	Réessayer après le délai ou ajuster <code>YAHRIACAD_AI_RATE_*</code>
Requêtes bloquées depuis le navigateur	Origine hors liste CORS	Ajouter l'origine à <code>YAHRIACAD_ALLOWED_ORIGINS</code>
Port 8080 occupé au démarrage	Process fantôme (vieux build)	<code>ss -tlnp grep 8080</code> puis tuer le process
Backend s'arrête immédiatement en prod	<code>APP_ENV=production</code> sans <code>JWT_SECRET</code>	Fail-fast voulu : fournir un secret (ou passer en dev)
WS refuse la connexion (400)	Handshake sans jeton quand JWT actif	Ajouter <code>?token=<jwt></code> à l'URL WebSocket
healthz : ai_model_loaded=false	Moteur up mais sans checkpoint	Normal au premier démarrage — recharger un .pt si besoin de RL

Variables d'environnement à connaître

VARIABLE	RÔLE (DÉFAUT)
YAHRIACAD_HTTP_PORT	Port HTTP du backend (8080)
YAHRIACAD_DB_URL	DSN PostgreSQL — vide = adaptateur mémoire
YAHRIACAD_AI_ADDR	Adresse gRPC du moteur IA (localhost:50051)
YAHRIACAD_JWT_SECRET / _AUTH_USERS	Active l'auth si un secret est présent (TTL 24h)
YAHRIACAD_ALLOWED_ORIGINS	CORS strict (Docker : http://localhost:3000)
YAHRIACAD_AI_RATE_RPS / _BURST	Limite des routes IA par IP (2 / 10)
YAHRIACAD_DATA_DIR	Répertoire des exports (./data)

Toute la configuration admet aussi un fichier JSON optionnel pointé par `YAHRIACAD_CONFIG` ; les variables d'environnement gagnent toujours. Pour la production, un reverse-proxy frontal est prêt à l'emploi : `configs/production/nginx.conf` centralise TLS, gzip et WebSocket.

10 · À VOS CARTES

Checklist de bout en bout

Le parcours complet, de zéro à la livraison — à cocher à chaque nouvelle carte :

- ☒ **Stack démarrée** `make docker-up`
Postgres, moteur IA, backend, frontend — les 4 conteneurs up.
- ☒ **Santé vérifiée** `GET /healthz`
status ok, ai_engine ok, database postgres.
- ☒ **Connexion** `login admin:admin`
JWT en localStorage ; « Continuer sans connexion » si auth off.
- ☒ **Projet créé + netlist importée**
Project Manager → import ; ERC vert dès le départ.
- ☒ **Placement IA lancé** `POST .../place`
Job asynchrone suivi à la barre de progression.
- ☒ **Routage global puis finitions** ⚡
`/route` pour la base, bouton ⚡ net par net pour finir.
- ☒ **Benchmark Arena** 🏆 joué
A* vs RL sur le board, verdict + ELO au leaderboard.
- ☒ **DRC + Auto-Healer + DFM verts**
Contrôles passés, violations triviales auto-corrigées.

- ✓ **Snapshot Time Machine capturé**
État figé avant livraison, diff possible plus tard.
- ✓ **Exports livrés gerber · bom · step · odbpp**
Archive .tgz ODB++ vérifiée, fichiers dans /data.

Mémo des commandes make

Cycle de dev **deps · build · lint**

`make deps` installe Go/pip/npm · `make build` compile backend + frontend · `make lint` passe gofmt, ruff et tsc · `make proto` régénère les stubs gRPC partagés.

Qualité **test · test-ai · test-frontend**

`make test` lance les suites Go et Python · `make test-frontend` exécute les E2E Playwright (auth tolérante aux trois modes) · CI : 3 jobs verts à chaque push.

IA & RL **models · train-router**

`make models` génère les checkpoints initiaux · `make train-router` `STEPS=2000000` entraîne le routeur PPO · reload à chaud via `POST /ai/model/reload`.

Cycle Docker **docker-build · up · down**

`make docker-build` reconstruit les images · `make docker-up` démarre la stack · `make docker-down` l'arrête en conservant les volumes pgdata/backenddata.

Observabilité express

`GET /metrics` expose les compteurs et histogrammes Prometheus (routes normalisées, cardinalité bornée) — branchez-y un Grafana ou un simple `curl | grep` pour suivre latences et erreurs. Un reverse-proxy TLS prêt à l'emploi attend dans `configs/production/nginx.conf`.

YahriaCad_

V0.1.0 · CI 3/3 VERTE

Backend Go hexagonal, moteur RL PyTorch à rechargement à chaud, collaboration CRDT, exports Gerber / ODB++ / STEP / BOM — le tout en une stack Docker.

github.com/assihervy-coder/YahriaCad

Rappel sécurité : le token GitHub personnel partagé dans l'historique de chat reste à révoquer depuis les paramètres développeur de GitHub, même après changement de dépôt.

Bon routage ⚡